

H5 SF2525

Tommaso Brambilla

brambi@kth.se

1 Part 0: Toy example

1.1 0.1

At first, we consider the iterations:

$$\begin{cases} \theta_0 = 1 \\ \theta_{n+1} = \theta_n - \frac{\partial f}{\partial \theta}(\theta_n, Y_n) \Delta t \quad n = 1, 2, \dots \end{cases}$$

The derivative with respect to θ of function $f(\theta_n, Y_n)$ can be written as:

$$\frac{\partial f}{\partial \theta}(\theta_n, Y_n) = 2(\theta_n - Y_n)$$

Plugging this into the update formula and simplifying, we obtain:

$$\theta_{n+1} = (1 - 2\Delta t)\theta_n + 2Y_n\Delta t$$

Now taking the expected value:

$$\mathbb{E}[|\theta_{n+1}|^2] = (1 - 2\Delta t)\mathbb{E}[|\theta_n|^2] + (2\Delta t)^2\mathbb{E}[Y_n^2] + \mathbb{E}[\theta_n Y_n]$$

Since all Y_n are i.i.d. standard normal variables (mean zero and variance one), we can conclude that $\mathbb{E}[Y_n^2] = 1$ and $\mathbb{E}[Y_n] = 0$. Also, since θ_n and Y_n are independent processes $\mathbb{E}[\theta_n Y_n] = 0$.

Setting $\alpha_n = \mathbb{E}[|\theta_n|^2]$ the equation becomes:

$$\alpha_{n+1} = (1 - 2\Delta t)^2\alpha_n + 4\Delta t^2$$

That is bounded if $0 < \Delta t < 1$. For finding C we search the steady state value so in other words we set $n \rightarrow \infty$ and we seek solution of:

$$\alpha = (1 - 2\Delta t)^2\alpha + 4\Delta t^2$$

that yields:

$$\alpha = \frac{\Delta t}{1 - \Delta t}$$

So we can conclude that the recursion is bounded by $C = \frac{\Delta t}{1 - \Delta t}$.

Note that this expression for C is only valid when $\Delta t > 0.5$, since in that case the process converges monotonically to the steady-state value as $n \rightarrow \infty$. If $\Delta t < 0.5$, then $C < 1$, which contradicts the fact that $\mathbb{E}[\theta_0^2] = 1$. Therefore, the correct upper bound is:

$$C = \max \left(1, \frac{\Delta t}{1 - \Delta t} \right)$$

Next, we aim to find the optimal solution to the following minimization problem:

$$\mathbb{E}[f(\theta_*, Y_n)] = \min_{\theta \in \mathbb{R}} \mathbb{E}[f(\theta_n, Y_n)]$$

Expanding the function that is defined as $f(\theta_n, Y_n) := |\theta_n - Y_n|^2$ and taking the expected value we can conclude that the minimization problem is clearly:

$$\min_{\theta \in \mathbb{R}} \mathbb{E}[\theta_n^2] + 1$$

that yields $\theta_* = 0$.

To search the convergence rate of $\lim_{n \rightarrow \infty} \mathbb{E}[|\theta_n - \theta_*|^2]$ we look for a n dependant function of θ_n . So we can rewrite the recursion as follows:

$$\alpha_{n+1} = (1 - 2\Delta t)^2 \alpha_n + (2\Delta t)^2$$

Where $\alpha_n = \mathbb{E}[|\theta^2|]$. This recursion can be written as:

$$\alpha_{n+1} = \lambda \alpha_n + c$$

and after applying recursion we get that the expression is:

$$\alpha_n \propto \lambda^n$$

So we can conclude that the convergence rate is $\mathcal{O}((1 + 2\Delta t)^{2n})$.

0.2 Matlab Code

```

1 N = 1e3;
2 dt = 0.3;
3
4 C = dt/(1-dt);
5 M = 1e5;
6
7 theta = zeros(M,N);
8 theta(:,1) = 1;
9 Y = sqrt(dt)*randn(M,N);
10 theta2 = zeros(M,N);
11 theta2(:, 1) = 1;
12
13 for j = 1:M
14
15     for i = 2:N
16
17         theta(j , i) = (1-2*dt).*theta(j , i-1) + 2*dt*Y(j , i);
18         theta2(j , i) = theta2(j , i)+ + theta(j,i).^2;
19
20     end
21
22 end

```

This code also compute $\mathbb{E}[\theta_n^2]$ and check if the C obtained in the previous exercise is correct. In order to do so, $M = 10^5$ realizations were computed.

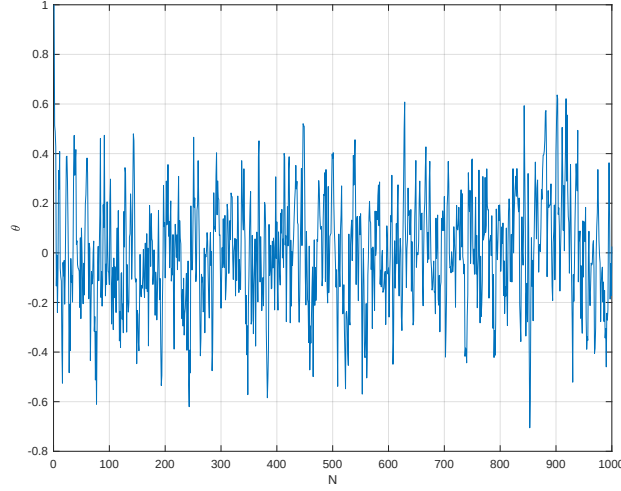


Figure 1: θ_n for $n = 1, 2, \dots, N$ with $\Delta t = 0.2$

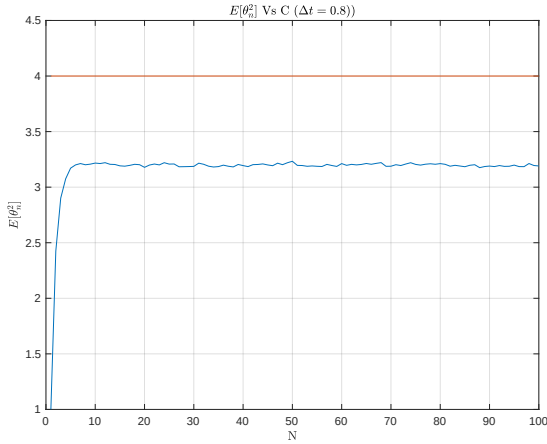


Figure 2: $\Delta t > 0.5$

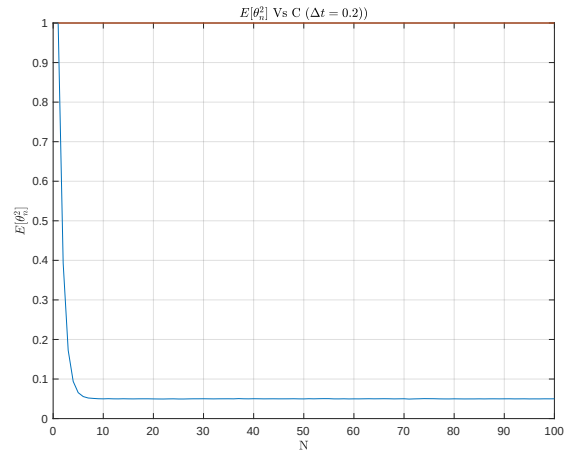


Figure 3: $\Delta t < 0.5$

In Figure 2 and 3 we can clearly see that $\mathbb{E}[\theta_n^2] \leq C$ for each iteration N , where the red line represents the C value.

0.3

Recall that an Ornstein-Uhlenbeck process is defined as follows:

$$dx_t = -\alpha x_t dt + \sigma dW_t$$

The forward Euler discretization for the equation is:

$$\Delta x_n = -\alpha x_n \Delta t + \sigma \Delta W_n$$

In our case the discretization is:

$$\theta_{n+1} = \theta_n - 2\Delta t \theta_n + 2Y_n \Delta t$$

that can be seen as:

$$\Delta\theta_n = -2\theta_n\Delta t + 2Y_n\Delta t$$

in order to match the last term and recalling that $Y_n \sim \mathcal{N}(0, 1)$ we can write that as $2\sqrt{\Delta t}(\sqrt{\Delta t}Y_n)$ so at the end we have an Ornstein-Uhlenbeck with $\alpha = 2$ and $\sigma = 2\sqrt{\Delta t}$.

Now compute the Gradient Descent method computing the expected value exactly.

$$\theta_{n+1} = \theta_n - \frac{\partial f}{\partial \theta}(\theta_n, Y_n)\Delta t$$

becomes:

$$\mathbb{E}[\theta_{n+1}] = \mathbb{E}[\theta_n] - 2\mathbb{E}[\theta_n]\Delta t \quad \rightarrow \quad \mathbb{E}[\theta_{n+1}] = (1 - 2\Delta t)\mathbb{E}[\theta_n]$$

For easy calculation substitute $\alpha_n = \mathbb{E}[\theta_n]$ and the recursion becomes:

$$\alpha_n = (1 - 2\Delta t)^n \alpha_0$$

Therefore, when computing the expected value directly, the convergence rate is exponential in n , in contrast to the previous case where the decay rate was exponential in $2n$.

Part 1: Minimization with TensorFlow

1. tf_mnist_lsq.py results

Using the original parameter settings, the network achieves an accuracy of 86%. Increasing K fourfold while keeping $\Delta t = 0.002$ (i.e., setting $K = 400$) improves the accuracy to 89%. A similar gain is observed by reducing the Mini-Batch size—halving it and using 5 Mini-Batches also results in 89% accuracy. Increasing the number of epochs to 30 yields a slightly lower, but still improved, accuracy of 88%. Overall, these results suggest that increasing the number of SDG steps tends to improve the performance of the algorithm.

2. tf_mnist_entropy.py results

With the given parameters, the algorithm achieves an accuracy of 91% on the MNIST training set. Replacing the activation function with *ReLU* increases the accuracy to 94%. Furthermore, switching the optimizer from SGD to Adam and doubling the value of M results in an accuracy of 95%.

Now, rewrite the α_0 for a network with 3 hidden layers ReLU activation function and K neurons. Recall that ReLU function is defined as $\text{ReLU}(z) = \max(0, z)$. For the hidden layer 1 we can write:

$$h_1^k = \theta_{1,k}^1 \text{ReLU}(\theta_{2,k}^1 \cdot x + \theta_{3,k}^1) \quad \text{for } k = 1, \dots, K$$

where $x \in \mathbb{R}^{784}$, $\theta_{1,k}^1, \theta_{3,k}^1 \in \mathbb{R}$ and $\theta_{2,k}^1 \in \mathbb{R}^{784}$. So for each neuron K in hidden layer 1 the data from the input layer are processed and the output will have dimension for a single training image $\mathbb{R}^K$.

Now, for the second hidden layer, the process is exactly the same:

$$h_2^k = \theta_{1,k}^2 \text{ReLU}(\theta_{2,k}^2 \cdot h_1^1 + \theta_{3,k}^2) \quad \text{for } k = 1, \dots, K$$

At this point, since the output from the first hidden layer is a vector of dimension K , it follows that $\theta_{2,k}^2 \in \mathbb{R}^K$. The procedure in the second hidden layer is therefore analogous to the first, with the main difference being the size of the weight matrices. The process begins with an input image represented as a vector of 784 pixels, which is passed through K neurons in the first hidden layer to produce a K -dimensional output. This output then serves as the input to the second hidden layer, which again consists of K neurons and applies the same

activation function and structure. The only significant difference appears in the output layer, where the weight matrix must map from $\mathbb{R}^K$ to $\mathbb{R}^{10}$, i.e., it has dimension $\mathbb{R}^{K \times 10}$.

So a suitable α_0 can be:

Given:

$$\begin{aligned} h_1^k &= \text{ReLU}(\theta_{2,k}^1 \cdot x + \theta_{3,k}^1) & \text{for } k = 1, \dots, K \\ h_2^k &= \text{ReLU}(\theta_{2,k}^2 \cdot h_1^k + \theta_{3,k}^2) & \text{for } k = 1, \dots, K \\ h_3^k &= \text{ReLU}(\theta_{2,k}^3 \cdot h_2^k + \theta_{3,k}^3) & \text{for } k = 1, \dots, K \end{aligned}$$

α_0 becomes:

$$\alpha_0(x, \theta) = \left(\sum_{k=1}^K \theta_{k,1}^4 h_k^3, \dots, \sum_{k=1}^K \theta_{k,10}^4 h_k^3 \right)$$

Here, $\theta_{k,i}^4$ denotes the weights of the output layer, forming a matrix of dimensions $K \times 10$. As a result,

$\alpha_0(x,$

$\theta)$ produces a vector in

$\mathbb{R}^{10}$ for a single input sample. This can be naturally extended to handle N training samples by evaluating the expression for each input in the batch.

Part 2: Minimization With Random Fourier Feature

1. One-Hot vector creation

The idea is to initialize a zero matrix of dimensions $N \times 10$, and then populate it according to the label values from the MNIST dataset.

```

1 M_OHE = zeros(N,10);
2
3
4 % Create a matrix of the labels using One-Hot-Encoding
5 for i = 1:1:N
6
7     val = labels(i);
8     M_OHE(i , val + 1) = 1;
9
10 end

```

The output matrix is a matrix where each row correspond to the label of each train image. Every value of the row is 0 besides the $val + 1$ position that is equal to 1. So we get a vector that has just one 1 in the position that represent the figure.

2.MATLAB Function

```

1 function [accuracy] = RandomFourierFeature(K , lambda)
2
3 load("MNISTDATA.mat")
4 load("valDATA.mat")
5
6 N = 60000;
7 M_OHE = zeros(N,10);

```

```

8 M_OHE_train = zeros(N/6 , 10);
9 imgs_vec = zeros(28*28 , N);
10 imgs_vec_train = zeros(28*28 , N/6);
11
12
13 % Create a matrix of the labels using One-Hot-Encoding
14 for i = 1:1:N
15
16     val = labels(i);
17     M_OHE(i , val + 1) = 1;
18
19
20 end
21
22 % Linearize the pixels matrix of the images
23 for i = 1:1:N
24
25     train_sample_k = imgs(: , : , i);
26     imgs_vec(: , i) = train_sample_k(:);
27
28 end
29
30 % Create a matrix of the labels using One-Hot-Encoding for train
31 for i = 1:1:N/6
32
33     val = labels_10k(i);
34     M_OHE_train(i , val + 1) = 1;
35
36 end
37
38 % Create wk sampled from multivariate distribution
39
40 sigma = 0.1;
41 d = 784;
42
43 wk_vec = sigma * randn(d, K);
44
45
46 % Build S matrix as  $S(n,k) = e^{i(wk) \cdot x_n}$ 
47
48 S = exp(1i * imgs_vec' * wk_vec);
49
50 % Initialize beta for every i
51
52 beta = zeros(K , 10);
53
54 % For every i solve:
55
56 for i = 1:1:10
57
58     beta(:, i) = (S' * S + lambda * size(imgs_vec,1) * eye(K)) \ (S' *
59         M_OHE(:, i));
60 end

```

```

61
62 % Test the results
63 S_test = exp(1i * imgs_vec_train' * wk_vec);
64 scores = real(S_test * beta);
65 [~, pred] = max(scores, [], 2);
66 accuracy = mean(pred-1 == labels_10k);

```

This function perform the minimization problem with Random Fourier Features. $\mathbf{K}$ and the regularization parameter λ are input variables. This function compute β with 60000 train images and test it on 10000 figures taken from MNIST database.

3. Test Function with different Ks

The function was tested with a fixed $\lambda = 10^{-3}$ and with $K = 2^{[1:11]}$ the following results were obtained:

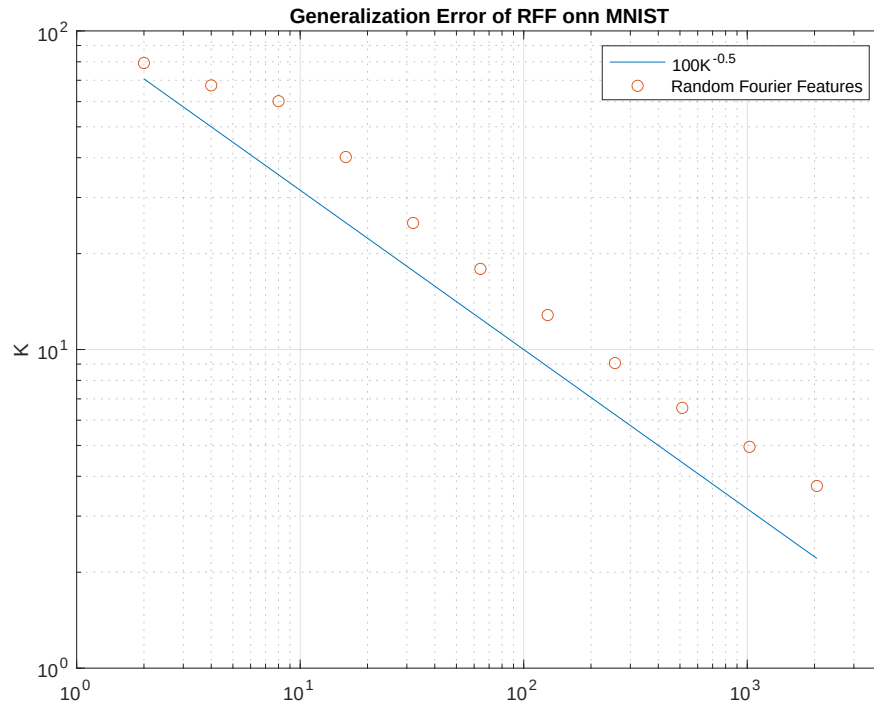


Figure 4: Error of RFF on MNIST

That follows similarly the plot shown in Exercise 4 file.

4. Fixed K varying λ

With fixed $K = 512$, different lambda were tested with RFF ($\lambda = 10^{[-6:0.1:2]}$)

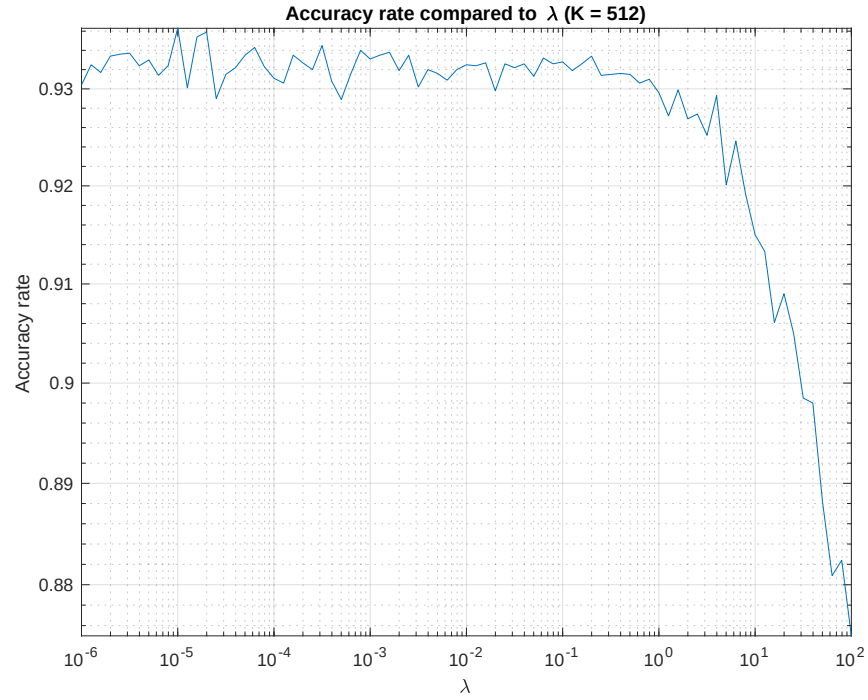


Figure 5: Behavior of Accuracy compared to λ

It is clearly visible that λ yields similar results of accuracy from 10^{-6} to 10^{-1} and then the accuracy start to decrease rapidly approaching and surpassing $\lambda = 1$

5. Alternative frequencies distribution

An alternative proposal for the frequency distribution is to use an anisotropic approach. In the standard (isotropic) setting, the variance σ of the multivariate distribution is assumed to be the same for all pixels. In contrast, the anisotropic variant assigns a variance to each pixel based on its empirical variance across the training set, scaled by the maximum observed variance. This approach gives more importance to pixels that vary the most—typically the central pixels in MNIST images—by assigning them higher variance when sampling the frequency vectors. As a result, pixels on the image borders, which tend to carry less information, are given lower variance.

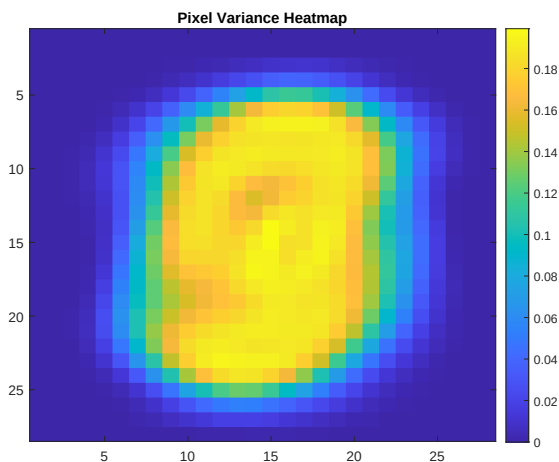


Figure 6: Pixels Variance

It is evident that the central pixels exhibit higher variance, indicating greater importance, while the variance near the edges is close to zero. By shaping the pixel-wise variance in this way, the algorithm is encouraged to focus more on the informative regions of the image. This effect is especially noticeable when using smaller values of K , where the difference in accuracy becomes more pronounced.